



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 02

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 04-09-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Actividades realizadas:

1.- Dibujar las iniciales de sus nombres, cada letra de un color diferente

```
28 // ---- Figuras ----
29 void CrearLetrasyFiguras()
30 {
31     GLfloat vertices_letras[] = {
32         // X Y Z R G B
33         // ----- LETRA S -----
34         // ----- MORADO BRILLANTE -----
35         // Superior
36         -0.97f, 0.8f, 0.0f, 0.6f, 0.2f, 1.0f,
37         -0.45f, 0.8f, 0.0f, 0.6f, 0.2f, 1.0f,
38         -0.97f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
39
40         -0.97f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
41         -0.45f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
42         -0.45f, 0.8f, 0.0f, 0.6f, 0.2f, 1.0f,
43
44         // Vertical superior
45         -0.97f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
46         -0.97f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
47         -0.82f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
48
49         -0.82f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
50         -0.82f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
51         -0.97f, 0.95f, 0.0f, 0.6f, 0.2f, 1.0f,
52
53         // Medio
54         -0.97f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
55         -0.45f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
56         -0.97f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
57
58         -0.97f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
59         -0.45f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
60         -0.45f, 0.55f, 0.0f, 0.6f, 0.2f, 1.0f,
61
62         // Vertical inferior
63         -0.45f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
64         -0.45f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
65         -0.60f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
66
67         -0.60f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
68         -0.60f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
69         -0.45f, 0.70f, 0.0f, 0.6f, 0.2f, 1.0f,
70
71         // Inferior
72         -0.97f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
73         -0.45f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
74         -0.97f, 0.45f, 0.0f, 0.6f, 0.2f, 1.0f,
75
76         -0.97f, 0.45f, 0.0f, 0.6f, 0.2f, 1.0f,
77         -0.45f, 0.45f, 0.0f, 0.6f, 0.2f, 1.0f,
78         -0.45f, 0.30f, 0.0f, 0.6f, 0.2f, 1.0f,
79
80         // ----- LETRA L -----
81         // ----- AQUA -----
82         // Vertical l
83         -0.35f, 0.45f, 0.0f, 0.0f, 1.0f, 1.0f,
84         -0.20f, 0.45f, 0.0f, 0.0f, 1.0f, 1.0f,
85         -0.35f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
86
87         -0.35f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
88         -0.20f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
89         -0.20f, 0.45f, 0.0f, 0.0f, 1.0f, 1.0f,
```

```

91 // Inferior _
92 -0.35f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
93 0.13f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
94 -0.35f, -0.07f, 0.0f, 0.0f, 1.0f, 1.0f,
95
96 -0.35f, -0.07f, 0.0f, 0.0f, 1.0f, 1.0f,
97 0.13f, -0.07f, 0.0f, 0.0f, 1.0f, 1.0f,
98 0.13f, -0.25f, 0.0f, 0.0f, 1.0f, 1.0f,
99
100 // ----- LETRA C -----
101 // ----- ANARANJADO -----
102 // Superior
103 0.30f, -0.25f, 0.0f, 1.0f, 0.0f, 0.8f,
104 0.85f, -0.25f, 0.0f, 1.0f, 0.0f, 0.8f,
105 0.85f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,
106
107 0.85f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,
108 0.30f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,
109 0.30f, -0.25f, 0.0f, 1.0f, 0.0f, 0.8f,
110
111 // Vertical
112 0.30f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,
113 0.30f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
114 0.45f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
115
116 0.45f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
117 0.45f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,
118 0.30f, -0.40f, 0.0f, 1.0f, 0.0f, 0.8f,

```

```

120 // Inferior
121 0.30f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
122 0.85f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
123 0.30f, -0.75f, 0.0f, 1.0f, 0.0f, 0.8f,
124
125 0.30f, -0.75f, 0.0f, 1.0f, 0.0f, 0.8f,
126 0.85f, -0.75f, 0.0f, 1.0f, 0.0f, 0.8f,
127 0.85f, -0.90f, 0.0f, 1.0f, 0.0f, 0.8f,
128
129 };
130
131 MeshColor* letras = new MeshColor();
132 letras->CreateMeshColor(vertices_letras, sizeof(vertices_letras) / sizeof(vertices_letras[0]));
133 meshColorList.push_back(letras);
134 }
135
136 void CreateShaders()
137 {
138     Shader* shader1 = new Shader(); // Para objetos con indices
139     shader1->CreateFromFiles(vShader, fShader);
140     shaderList.push_back(*shader1);
141
142     Shader* shader2 = new Shader(); // Para color directo en el VAO
143     shader2->CreateFromFiles(vShaderColor, fShaderColor);
144     shaderList.push_back(*shader2);
145 }

```

```

147     int main()
148     {
149         mainWindow = Window(800, 600);
150         mainWindow.Initialise();
151
152         CrearLetrasyFiguras();
153         CreateShaders();
154
155         GLuint uniformProjection = 0;
156         GLuint uniformModel = 0;
157
158         // Creando la matriz de proyección para la vista ortográfica (2D)
159         glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f);
160
161         glEnable(GL_DEPTH_TEST);
162
163         while (!mainWindow.getShouldClose())
164         {
165             glfwPollEvents();
166
167             glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
168             glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
169
170             // ----- Dibujar letras con color -----
171             shaderList[1].useShader();
172             uniformModel = shaderList[1].getModelLocation();
173             uniformProjection = shaderList[1].getProjectLocation();
174
175             glm::mat4 model(1.0f);
176             model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
177             model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
178
179             glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
180             glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
181
182             meshColorList[0] -> RenderMeshColor();
183
184             glUseProgram(0);
185             mainWindow.swapBuffers();
186         }
187         return 0;
188     }

```

```

while (!mainWindow.getShouldClose())
{
    glfwPollEvents();

    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // ----- Dibujar letras con color -----
    shaderList[1].useShader();
    uniformModel = shaderList[1].getModelLocation();
    uniformProjection = shaderList[1].getProjectLocation();

    glm::mat4 model(1.0f);
    model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
    model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));

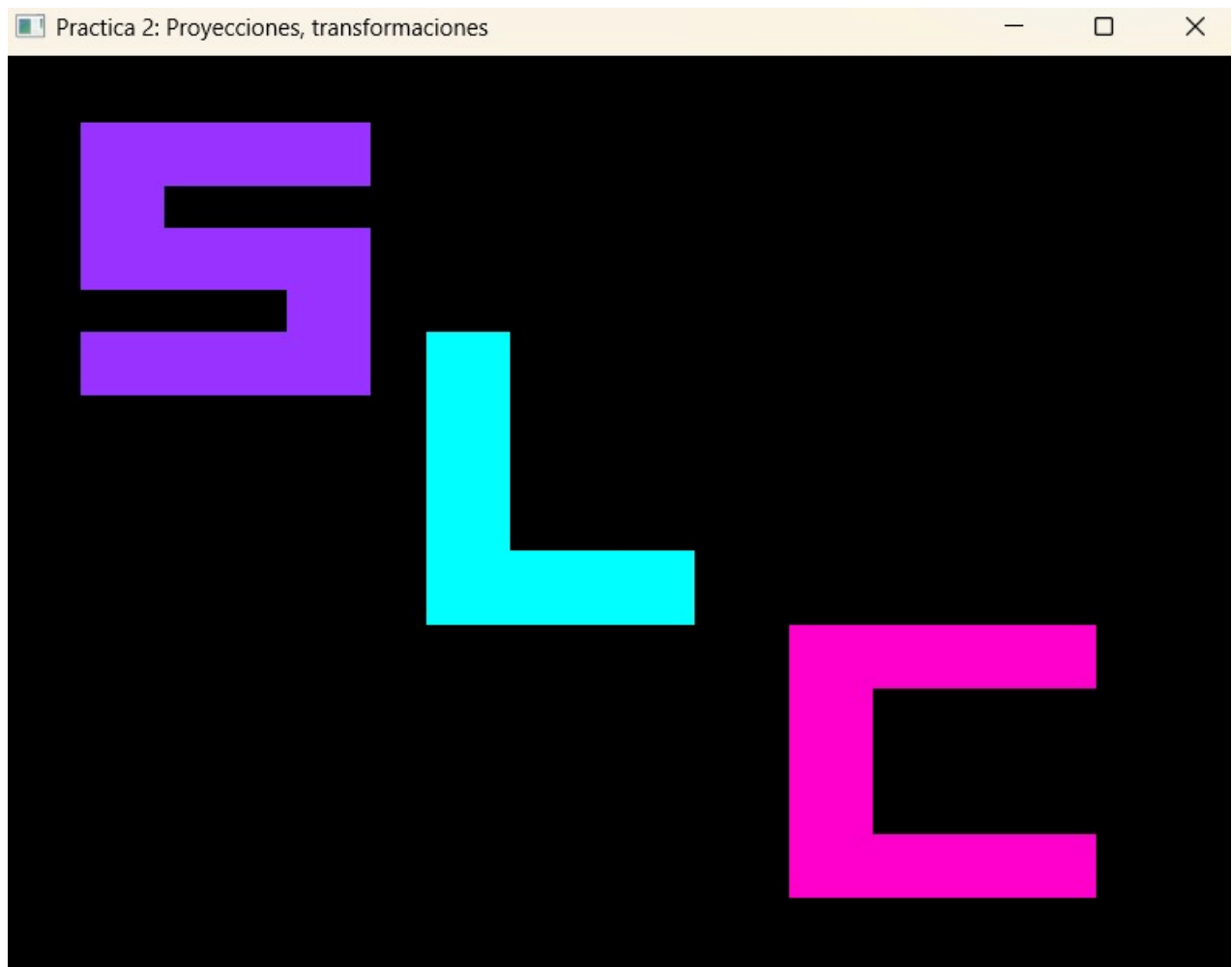
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

    meshColorList[0] -> RenderMeshColor();

    glUseProgram(0);
    mainWindow.swapBuffers();
}
return 0;

```

El ejecutable que obtenemos es el siguiente:



2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triangulos y cuadrados será instanciando piramides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp.

```

1 //Práctica 2: índices, mesh, proyecciones, transformaciones geométricas
2 #include <stdio.h>
3 #include <string.h>
4 #include <cmath>
5 #include <vector>
6 #include <glew.h>
7 #include <glfw3.h>
8 //glm
9 #include <glm.hpp>
10 #include <gtc\matrix_transform.hpp>
11 #include <gtc\type_ptr.hpp>
12 //clases para dar orden y limpieza al código
13 #include "Mesh.h"
14 #include "Shader.h"
15 #include "Window.h"
16 //Dimensiones de la ventana
17 const float toRadians = 3.14159265f / 180.0; //grados a radianes
18 Window mainWindow;
19 std::vector<Mesh*> meshList;
20 std::vector<MeshColor*> meshColorList;
21 std::vector<Shader*> shaderList;
22 //Vertex Shader
23 static const char* vShader = "shaders/shader.vert";
24 static const char* fShader = "shaders/shader.frag";
25 static const char* vShaderColor = "shaders/shadercolor.vert";
26 static const char* fShaderColor = "shaders/shadercolor.frag";
27 //shaders nuevos se crearían acá
28 //----- SHADERS NUEVOS -----
29 static const char* fRojo = "shaders/rojo.frag";
30 static const char* fVerde = "shaders/verde.frag";
31 static const char* fAzul = "shaders/azul.frag";
32 static const char* fCafe = "shaders/cafe.frag";
33 static const char* fVerdeObs = "shaders/verdeObs.frag";
34
35 float angulo = 0.0f;
36 //color café en RGB : 0.478, 0.255, 0.067
37
38 //Pirámide triangular regular
39 void CreaPiramide()
40 {
41     unsigned int indices[] = {
42         0,1,2,
43         1,3,2,
44         3,0,2,
45         1,0,3
46     };
47
48     GLfloat vertices[] = {
49         -0.75f, -0.5f, 0.0f, //0
50         0.75f, -0.5f, 0.0f, //1
51         0.0f, 0.5f, -0.25f, //2
52         0.0f, -0.5f, -0.5f, //3
53     };
54
55     Mesh* obj1 = new Mesh();
56     obj1->CreateMesh(vertices, indices, 12, 12);
57     meshList.push_back(obj1);
58 }

```

```

60 //Vértices de un cubo
61 void CrearCubo()
62 {
63     unsigned int cubo_indices[] = {
64         // front
65         0, 1, 2,
66         2, 3, 0,
67         // right
68         1, 5, 6,
69         6, 2, 1,
70         // back
71         7, 6, 5,
72         5, 4, 7,
73         // left
74         4, 0, 3,
75         3, 7, 4,
76         // bottom
77         4, 5, 1,
78         1, 0, 4,
79         // top
80         3, 2, 6,
81         6, 7, 3
82     };
83
84     GLfloat cubo_vertices[] = {
85         // front
86         -0.6f, -0.7f, 0.5f,
87         0.6f, -0.7f, 0.5f,
88         0.6f, 0.5f, 0.5f,
89         -0.6f, 0.5f, 0.5f,
90
91         // back
92         -0.6f, -0.7f, -0.5f,
93         0.6f, -0.7f, -0.5f,
94         0.6f, 0.5f, -0.5f,
95         -0.6f, 0.5f, -0.5f
96     };
97     Mesh* cubo = new Mesh();
98     cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
99     meshList.push_back(cubo);
100 }
101
240 void CreateShaders()
241 {
242     //0
243     // Shader estándar para mallas usando indices
244     Shader* shader1 = new Shader();
245     shader1->CreateFromFiles(vShader, fShader);
246     shaderList.push_back(*shader1);
247
248     //1
249     // Shader que incluye color en los vértices
250     Shader* shader2 = new Shader();
251     shader2->CreateFromFiles(vShaderColor, fShaderColor);
252     shaderList.push_back(*shader2);
253
254     //2
255     // Shaders específicos para diferentes colores
256     Shader* shaderRojo = new Shader();
257     shaderRojo->CreateFromFiles("shaders/shader.vert", "shaders/rojo.frag");
258     shaderList.push_back(*shaderRojo);
259
260     //3
261     Shader* shaderVerdeObs = new Shader();
262     shaderVerdeObs->CreateFromFiles("shaders/shader.vert", "shaders/verdeObs.frag");
263     shaderList.push_back(*shaderVerdeObs);
264
265     //4
266     Shader* shaderAzul = new Shader();
267     shaderAzul->CreateFromFiles("shaders/shader.vert", "shaders/azul.frag");
268     shaderList.push_back(*shaderAzul);
269
270     //5
271     Shader* shaderCafe = new Shader();
272     shaderCafe->CreateFromFiles("shaders/shader.vert", "shaders/cafe.frag");
273     shaderList.push_back(*shaderCafe);

```

```

275 //6
276 Shader* shaderVerde = new Shader();
277 shaderVerde->CreateFromFiles("shaders/shader.vert", "shaders/verde.frag");
278 shaderList.push_back(*shaderVerde);
279
280
281 /*
282 Shader* shader2 = new Shader();//shader para usar color como parte del VAO: letras
283 shader2->CreateFromFiles(vShaderColor, fShaderColor);
284 shaderList.push_back(*shader2); */
285
286 int main()
287 {
288     mainWindow = Window(800, 600);
289     mainWindow.Initialise();
290     CreaPiramide(); //índice 0 en MeshList
291     CrearCubo(); //índice 1 en MeshList
292     CrearLetrasyFiguras(); //usa MeshColor, índices en MeshColorList
293     CreateShaders();
294     GLuint uniformProjection = 0;
295     GLuint uniformModel = 0;
296     //Projection: Matriz de Dimensión 4x4 para indicar si vemos en 2D( orthogonal) o en 3D) perspectiva
297     glm::mat4 projection = glm::ortho(-1.0f, 1.0f, -1.0f, 1.0f, 0.1f, 100.0f);
298     //glm::mat4 projection = glm::perspective(glm::radians(60.0f) ,mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);
299
300     //Model: Matriz de Dimensión 4x4 en la cual se almacena la multiplicación de las transformaciones geométricas.
301     glm::mat4 model(1.0); //fuera del while se usa para inicializar la matriz con una identidad

```

```

303 //Loop mientras no se cierra la ventana
304 while (!mainWindow.getShouldClose())
305 {
306     //Recibir eventos del usuario
307     glfwPollEvents();
308     //Limpiar la ventana
309     glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
310     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); //Se agrega limpiar el buffer de profundidad
311
312     /*// ----- Dibujar letras con color -----
313     shaderList[1].useShader();
314     uniformModel = shaderList[1].getModelLocation();
315     uniformProjection = shaderList[1].getProjectLocation();
316
317     glm::mat4 model(1.0f);
318     model = glm::translate(model, glm::vec3(0.0f, 0.0f, -2.0f));
319     model = glm::scale(model, glm::vec3(0.9f, 0.9f, 0.9f));
320
321     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
322     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
323
324     meshColorList[0]->RenderMeshColor(); */
325
326     //----- PIRAMIDE AZUL -----
327     shaderList[4].useShader();
328     uniformModel = shaderList[4].getModelLocation();
329     uniformProjection = shaderList[4].getProjectLocation();
330     angulo += 0.01;
331     //Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométricas
332     model = glm::mat4(1.0);
333     model = glm::translate(model, glm::vec3(0.0f, 0.2f, -3.0f));
334     model = glm::scale(model, glm::vec3(0.60f, 0.55f, 0.6f));
335     glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
336     glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

```



```

336 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
337 meshList[0]->RenderMesh();
338
339 //----- CUBO ROJO -----
340 shaderList[2].useShader();
341 uniformModel = shaderList[2].getModelLocation();
342 uniformProjection = shaderList[2].getProjectLocation();
343 angulo += 0.01;
344 //Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométricas
345 model = glm::mat4(1.0);
346 model = glm::translate(model, glm::vec3(0.0f, -0.42f, -3.0f));
347 model = glm::scale(model, glm::vec3(0.6f, 0.69f, 0.6f));
348 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
349 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
350 meshList[1]->RenderMesh();
351
352 //----- VENTANA IZQUIERDA -----
353 shaderList[6].useShader();
354 uniformModel = shaderList[6].getModelLocation();
355 uniformProjection = shaderList[6].getProjectLocation();
356 angulo += 0.01;
357 //Inicializar matriz de dimensión 4x4 que servirá como matriz de modelo para almacenar las transformaciones geométricas
358 model = glm::mat4(1.0);
359 model = glm::translate(model, glm::vec3(-0.17f, -0.25f, -2.0f));
360 model = glm::scale(model, glm::vec3(0.18f, 0.23f, 0.18f));
361 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
362 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
363 meshList[1]->RenderMesh();

```

```

365 //----- VENTANA DERECHA -----
366 shaderList[6].useShader();
367 uniformModel = shaderList[6].getModelLocation();
368 uniformProjection = shaderList[6].getProjectLocation();
369 angulo += 0.01;
370 model = glm::mat4(1.0);
371 model = glm::translate(model, glm::vec3(0.17f, -0.25f, -2.0f));
372 model = glm::scale(model, glm::vec3(0.18f, 0.23f, 0.18f));
373 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
374 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
375 meshList[1]->RenderMesh();
376
377 //----- PUERTA -----
378 shaderList[6].useShader();
379 uniformModel = shaderList[6].getModelLocation();
380 uniformProjection = shaderList[6].getProjectLocation();
381 angulo += 0.01;
382 model = glm::mat4(1.0);
383 model = glm::translate(model, glm::vec3(0.0f, -0.68f, -2.0f));
384 model = glm::scale(model, glm::vec3(0.18f, 0.32f, 0.18f));
385 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
386 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
387 meshList[1]->RenderMesh();

```

```

389 //----- TRONCO IZQUIERDO -----
390 shaderList[5].useShader();
391 uniformModel = shaderList[5].getModelLocation();
392 uniformProjection = shaderList[5].getProjectLocation();
393 angulo += 0.01;
394 model = glm::mat4(1.0);
395 model = glm::translate(model, glm::vec3(-0.7f, -0.65f, -2.0f));
396 model = glm::scale(model, glm::vec3(0.18f, 0.3f, 0.18f));
397 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
398 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
399 meshList[1]->RenderMesh();
400
401 //----- TRONCO DERECHO -----
402 shaderList[5].useShader();
403 uniformModel = shaderList[5].getModelLocation();
404 uniformProjection = shaderList[5].getProjectLocation();
405 angulo += 0.01;
406 model = glm::mat4(1.0);
407 model = glm::translate(model, glm::vec3(0.7f, -0.65f, -2.0f));
408 model = glm::scale(model, glm::vec3(0.18f, 0.3f, 0.18f));
409 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
410 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
411 meshList[1]->RenderMesh();

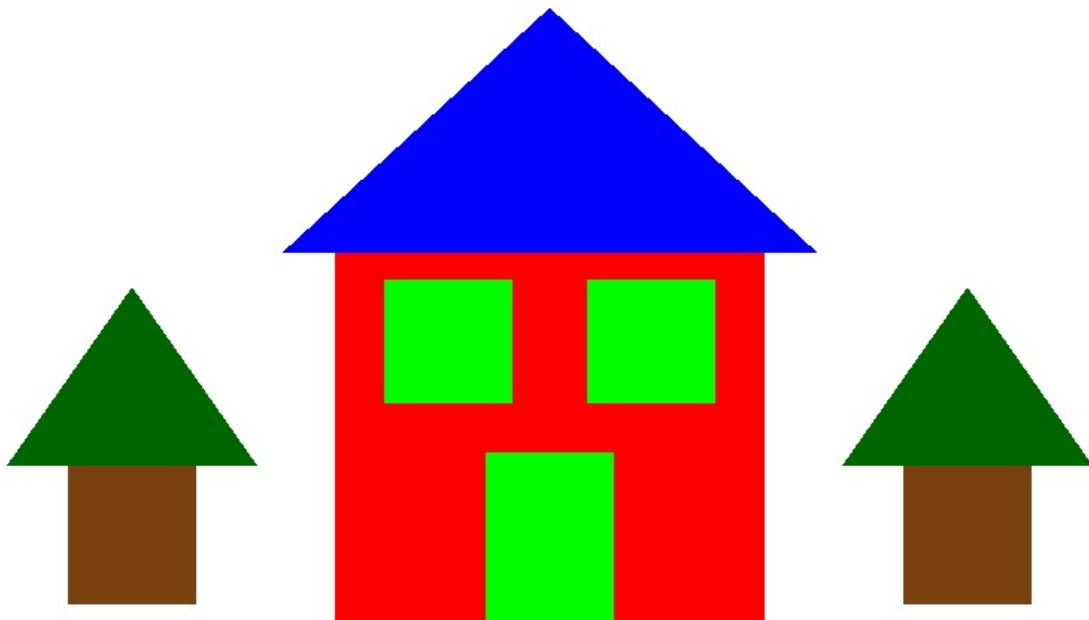
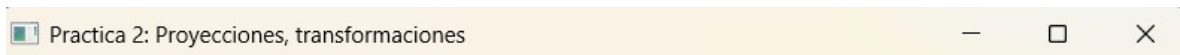
```

```

413 //----- ARBOL IZQUIERDO -----
414 shaderList[3].useShader();
415 uniformModel = shaderList[3].getModelLocation();
416 uniformProjection = shaderList[3].getProjectLocation();
417 angulo += 0.01;
418 model = glm::mat4(1.0);
419 model = glm::translate(model, glm::vec3(-0.7f, -0.35f, -1.0f));
420 model = glm::scale(model, glm::vec3(0.28f, 0.4f, 0.28f));
421 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
422 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
423 meshList[0] -> RenderMesh();
424
425 //----- ARBOL DERECHO -----
426 shaderList[3].useShader();
427 uniformModel = shaderList[3].getModelLocation();
428 uniformProjection = shaderList[3].getProjectLocation();
429 angulo += 0.01;
430 model = glm::mat4(1.0);
431 model = glm::translate(model, glm::vec3(0.7f, -0.35f, -1.0f));
432 model = glm::scale(model, glm::vec3(0.28f, 0.4f, 0.28f));
433 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model)); //FALSE ES PARA QUE NO SEA TRANSPUESTA
434 glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
435 meshList[0] -> RenderMesh();
436
437 glUseProgram(0);
438 mainWindow.swapBuffers();
439
440 }
441 return 0;
442 }

```

El ejecutable que obtenemos es el siguiente:



Problemas presentados:

Tuve un poco de problemas al cambiar el color de las letras, fue algo muy simple y solo fue que yo me estaba revolviendo en mucho código, realmente solo fue poner bien la estructura.

También tuve problemas al agregar los shaders para los diferentes colores, no entendía bien en donde tenía que agregar los archivos y con que estructura ponerla. Las coordenadas ya las pude entender y ya me salen muy bien.

Conclusión:

- a. Los ejercicios del reporte: Tuvieron un poco de complejidad, pero siempre va al ritmo de lo que vemos en clase.
- b. Comentarios generales: Creo que, si faltó poner más comentarios o darlos en clase, comentarios más específicos. El ritmo de la clase fue bueno, se explica a detalle un poco rápido y a veces podemos perdernos, pero creo que si no va rápido no se termina de explicar todo.
- c. Conclusión: La práctica me gustó, su complejidad esta interesante ya que, si te hace pensar y buscar como solucionarlo, pero la verdad tuve ayuda de IA, ya que en alguna página no encontraba como tal que hacer para solucionar mis errores.

Bibliografía en formato APA

-Mikejo5000. (2024, March 11). Use Shader Designer to create custom visual effects - Visual Studio (Windows). Microsoft.com. <https://learn.microsoft.com/en-us/visualstudio/designers/shader-designer?view=vs-2022>